<!-- به نام خدا -->

مهندسی اینترنت، جلسه صفر: مقدمه

اینترنت چطور به اینجا رسید؟

داستان مسئله‌ها و تصمیم‌های مهندسی که اینترنت امروز را ساخته‌اند

محمد بصیرزاده

دانشگاه شهید چمران اهواز، دانشکدهٔ مهندسی، پاییز ۱۴۰۵

سناریوی واقعی: یک نشانی را تا چه مدت به خاطر بسپاریم؟

فرض کنید مسئول DNS پایگاه دادهٔ DynamoDB در Amazon هستید. DNS دفترچهٔ نشانی اینترنت است و نام هر سرویس را به نشانی سرورهایش تبدیل می‌کند (جلسهٔ ۱). این سرورها پیوسته تغییر می‌کنند؛ برای همین هر کلاینت پاسخ DNS را فقط تا مدتی که شما تعیین می‌کنید نگه می‌دارد و پس از آن دوباره می‌پرسد. به این نگه‌داری cache و به آن مدت TTL می‌گویند. **دو خطر را با هم بسنجید**: سرور خراب تا چه مدت در فهرست باقی می‌ماند، و اگر خود DNS دچار خطا شود چقدر فرصت دارید؟

الف) ۵ ثانیه	**ب)** ۵ دقیقه	**ج)** یک ساعت	**د)** یک روز

سناریوی واقعی: یک نشانی را تا چه مدت به خاطر بسپاریم؟

الف) ۵ ثانیه	ب) ۵ دقیقه	ج) یک ساعت	د) یک روز

انتخاب Amazon: الف) اما ۲۸ مهر ۱۴۰۴ همین تصمیم، در کنار دو تصمیم دیگر، هزاران سایت را مختل کرد:

تصمیم	چرا منطقی بود	پیامد در ۲۸ مهر ۱۴۰۴
TTL فقط ۵ ثانیه	حذف فوری سرور خراب از فهرست	**فهرست خالی هم در چند ثانیه به همه رسید**
به‌روزرسانی خودکار DNS با سه نسخه	صدها هزار رکورد، بدون خطای انسانی	دو نسخه تداخل کردند و نشانی‌ها حذف شد
وابستگی به یک منطقهٔ Amazon	ارزان‌تر و ساده‌تر از چند منطقه	قطعی هم‌زمان Signal و Zoom و Duolingo

هر سه تصمیم منطقی بود و هرکدام پیامدی داشت؛ این جلسه داستان همین تصمیم‌هاست.

ARPANET و پیام «LO»

۱۹۶۹ ۱۳۴۸ — ۱۹۸۳ ۱۳۶۱ — ۱۹۹۱ ۱۳۷۰ — ۱۹۹۳ ۱۳۷۱ — ۱۹۹۵ ۱۳۷۴ — ۲۰۰۴ ۱۳۸۳ — ۲۰۱۵ ۱۳۹۴ — ۲۰۲۲ ۱۴۰۱

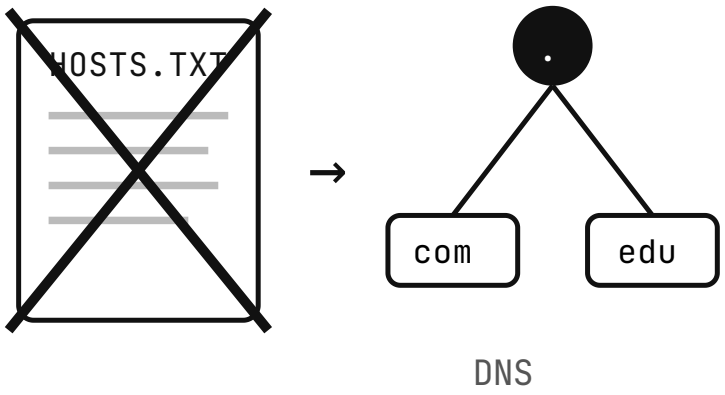


اولین پیام شبکه قرار بود LOGIN باشد؛ سیستم بعد از دو حرف از کار افتاد و «LO» اولین پیام ARPANET شد؛ شبکه‌ای برای اشتراک کامپیوترهای گران، نه برای جنگ.

در این درس زنجیره‌ای که هر حلقه‌اش می‌تواند وسط کار بشکند

TCP/IP و DNS

۱۹۶۹ ۱۳۴۸ — **۱۹۸۳** ۱۳۶۱ — ۱۹۹۱ ۱۳۷۰ — ۱۹۹۳ ۱۳۷۱ — ۱۹۹۵ ۱۳۷۴ — ۲۰۰۴ ۱۳۸۳ — ۲۰۱۵ ۱۳۹۴ — ۲۰۲۲ ۱۴۰۱

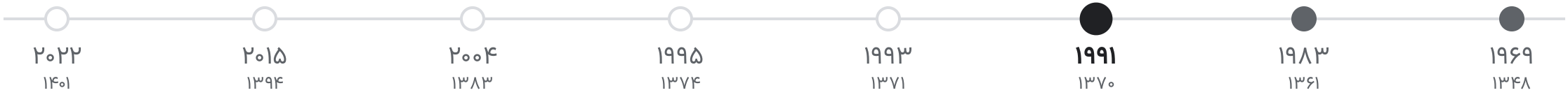


DNS

در یکم ژانویهٔ ۱۹۸۳ همهٔ ARPANET یک‌شبه به TCP/IP کوچ کرد، و در همان سال DNS جای فایل مشترکی به نام HOSTS.TXT را گرفت.

در این درس DNS در جلسهٔ ۱ و TCP در جلسهٔ ۴

وب متولد می‌شود





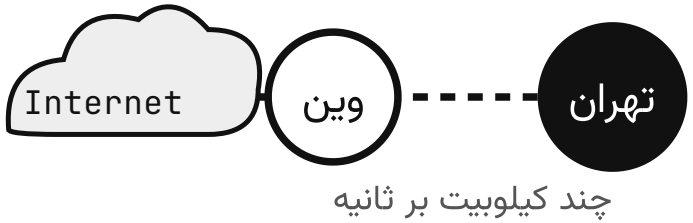
تیم برنرز-لی در CERN سه چیز را با هم ساخت: URL برای نام‌گذاری، HTTP برای گفتگو و HTML برای نمایش. نخستین وب‌سایت هنوز روی info.cern.ch باز می‌شود.

در این درس URL در جلسهٔ ۱ و HTTP در جلسهٔ ۲

ایران به اینترنت وصل می‌شود

۱۹۶۹ ۱۳۴۸ — ۱۹۸۳ ۱۳۶۱ — ۱۹۹۱ ۱۳۷۰ — **۱۹۹۳ ۱۳۷۱** — ۱۹۹۵ ۱۳۷۴ — ۲۰۰۴ ۱۳۸۳ — ۲۰۱۵ ۱۳۹۴ — ۲۰۲۲ ۱۴۰۱

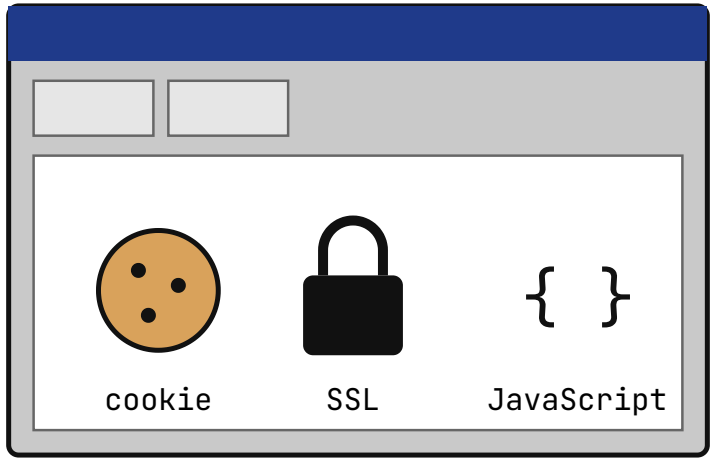
در دی ۱۳۷۱ پژوهشگاه دانش‌های بنیادی (IPM) با یک خط ۹۶۰۰ baud به دانشگاه وین وصل شد؛ نخست فقط برای ایمیل، و سپس اتصال کامل اینترنت با ۵۰۰ نشانی IP.



در این درس مسیر بین‌الملل و DNS داخلی هنوز مسئلهٔ مهندسی ماست

جنگ اول مرورگرها

۱۹۶۹ ۱۳۴۸ — ۱۹۸۳ ۱۳۶۱ — ۱۹۹۱ ۱۳۷۰ — ۱۹۹۳ ۱۳۷۱ — **۱۹۹۵ ۱۳۷۴** — ۲۰۰۴ ۱۳۸۳ — ۲۰۱۵ ۱۳۹۴ — ۲۰۲۲ ۱۴۰۱

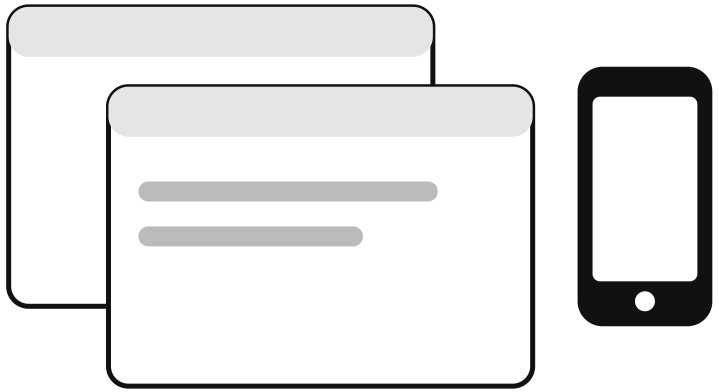


Netscape در دو سال کوکی و SSL و JavaScript را ساخت و Microsoft با Internet Explorer رایگان جواب داد؛ در اوایل دههٔ ۲۰۰۰ سهم IE از بازار از ۹۰ درصد گذشت.

در این درس کوکی در جلسهٔ ۳، و TLS که نوادهٔ SSL است در جلسهٔ ۴

جنگ دوم مرورگرها و وب همراه

۱۹۶۹ ۱۳۴۸ — ۱۹۸۳ ۱۳۶۱ — ۱۹۹۱ ۱۳۷۰ — ۱۹۹۳ ۱۳۷۱ — ۱۹۹۵ ۱۳۷۴ — **۲۰۰۴ ۱۳۸۳** — ۲۰۱۵ ۱۳۹۴ — ۲۰۲۲ ۱۴۰۱

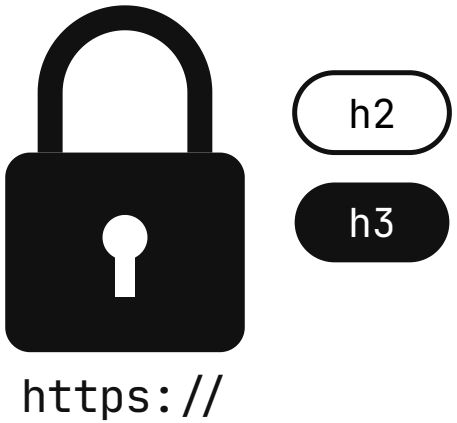


Firefox (۲۰۰۴) و Chrome (۲۰۰۸) انحصار را شکستند؛ سازندگان مرورگر WHATWG را تشکیل دادند تا استاندارد را از روی رفتار واقعی مرورگرها بنویسند، و iPhone (۲۰۰۷) وب را به جیب آورد.

در این درس استاندارد URL در جلسهٔ ۱ کار همین WHATWG است

وب امن و سریع

۱۹۶۹ ۱۳۴۸ — ۱۹۸۳ ۱۳۶۱ — ۱۹۹۱ ۱۳۷۰ — ۱۹۹۳ ۱۳۷۱ — ۱۹۹۵ ۱۳۷۴ — ۲۰۰۴ ۱۳۸۳ — **۲۰۱۵ ۱۳۹۴** — ۲۰۲۲ ۱۴۰۱

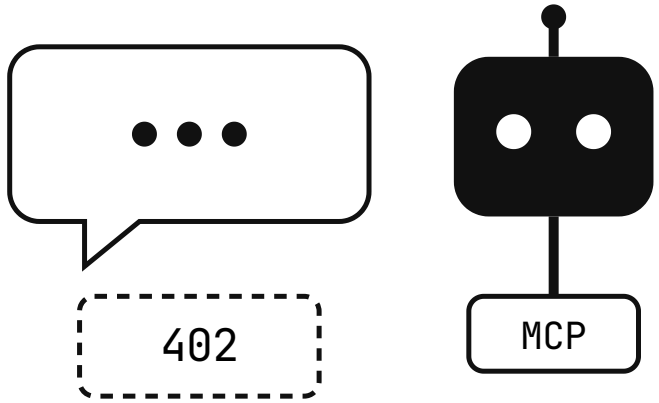


Let's Encrypt گواهی HTTPS را رایگان و خودکار کرد؛ HTTP/2 در ۲۰۱۵ و HTTP/3 روی QUIC در ۲۰۲۲ پروتکل را برای وب امروز بازنویسی کردند.

در این درس HTTP/2 و HTTP/3 و TLS در جلسهٔ ۴

عصر هوش مصنوعی

۱۹۶۹ ۱۳۴۸ — ۱۹۸۳ ۱۳۶۱ — ۱۹۹۱ ۱۳۷۰ — ۱۹۹۳ ۱۳۷۱ — ۱۹۹۵ ۱۳۷۴ — ۲۰۰۴ ۱۳۸۳ — ۲۰۱۵ ۱۳۹۴ — **۲۰۲۲** ۱۴۰۱



ChatGPT (آذر ۱۴۰۱) گفتگو با مدل‌ها را همه‌گیر کرد؛ حالا agentها خودشان وب را می‌خوانند و با MCP به ابزارها وصل می‌شوند، و کد وضعیت ۴۰۲ ابزار پرداخت agentها شده است.

در این درس کلاینت‌های ماشینی و MCP در جلسهٔ ۷

هر ایستگاه یک تصمیم است؛ در ادامهٔ جلسه چند تا از آن‌ها را از نزدیک بررسی می‌کنیم.

بخش ۱ از ۷

شبکهٔ شبکه‌ها

تا اینجا: دهه‌های ۱۹۶۰ و ۱۹۷۰؛ هنوز نه وبی بود، نه مرورگری.

سؤال این بخش

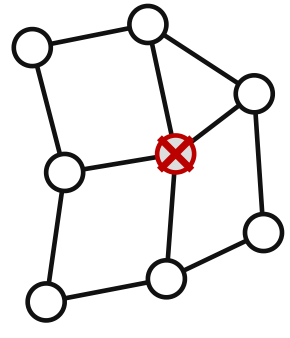
چطور کامپیوترها و شبکه‌های ناهمگون را به هم وصل کنیم، بی‌آنکه یک مرکز همه را اداره کند؟

00-how-we-got-here.todo

- [x] ۱۹۶۹ تا ۱۹۸۳ ◀
- [ ] ۱۹۸۹ تا ۱۹۹۳
- [ ] ۱۹۹۴ تا ۲۰۰۱
- [ ] ۲۰۰۴ تا ۲۰۱۲
- [ ] ۲۰۱۱ تا ۲۰۲۲
- [ ] ۲۰۲۲ تا امروز
- [ ] از اینجا به بعد

چرا شبکه‌ای که هیچ مرکزی ندارد؟

- دههٔ ۱۹۶۰: پل باران برای بقا در جنگ، شبکهٔ توزیع‌شده را پیشنهاد داد و دیویس نامش را packet گذاشت
- (۱۹۶۹) ARPANET این ایده را برای اشتراک کامپیوترهای گران به کار برد
- ۱۹۷۴: Cerf و Kahn با TCP/IP شبکه‌ها را به هم وصل کردند: **شبکهٔ شبکه‌ها**؛ شبکه فقط بسته می‌رساند و هوشمندی در دو سر است (end-to-end)



متمرکز: یک خرابی کافی است

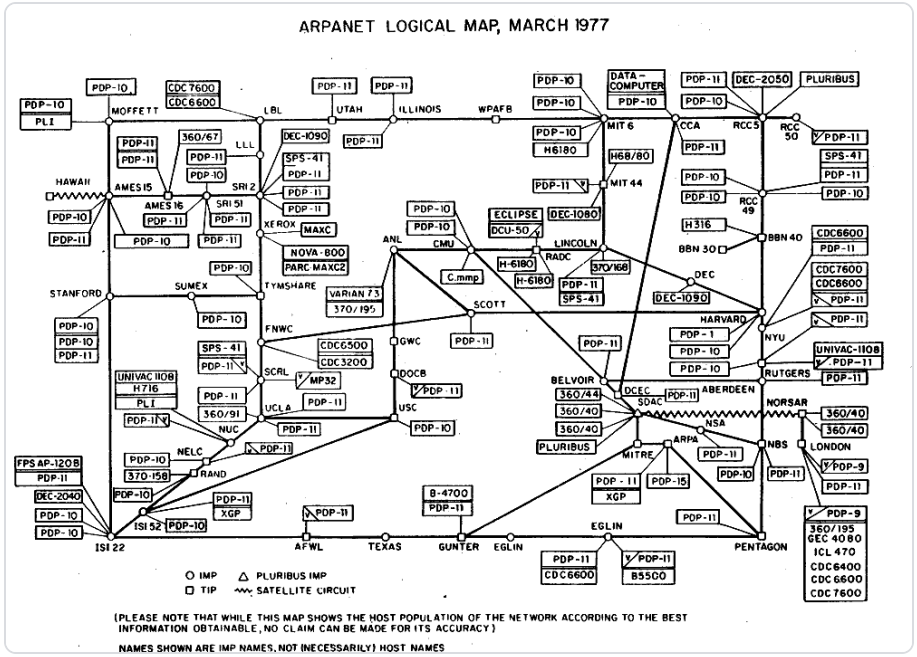
توزیع‌شده: راه دیگری هست

شبکهٔ ساده و دو سرِ هوشمند: برای همین هر برنامهٔ تازه، از وب تا agent، بدون اجازهٔ شبکه روی اینترنت ساخته شد.

۱۹۶۹ تا ۱۹۸۳

چطور یک شبکهٔ کامل را در یک شب عوض کنیم؟

- در یکم ژانویهٔ ۱۹۸۳ همهٔ ARPANET باید هم‌زمان به TCP/IP کوچ می‌کرد؛ **هر میزبانی که آماده نبود، قطع می‌شد**
- تاریخ کوچ را RFC 801، یکی از سندهای رسمی اینترنت، از نوامبر ۱۹۸۱ اعلام کرده بود؛ بیش از یک سال فرصت برای همه
- در همان سال DNS طراحی شد تا جای HOSTS.TXT را بگیرد، فهرست مشترکی از نام‌ها و نشانی‌ها که همه دستی کپی‌اش می‌کردند؛ همان DNS که در قطعی AWS یک نامش پاک شد (جلسهٔ ۱)



کل ARPANET در مارس ۱۹۷۷ در یک صفحه جا می‌شد

منبع: ARPANET · Public domain

وقتی یک فهرست مرکزی دیگر مقیاس ندارد، باید توزیعش کرد؛ DNS همین تصمیم بود.

چرا IPv6 را نمی‌شود مثل ۱۹۸۳ یک‌شبه جایگزین کرد؟

IPv6، نسخهٔ تازهٔ نشانی‌های اینترنت، از ۱۹۹۸ استاندارد است؛ سهمش در آمار Google تازه اردیبهشت ۱۴۰۵ به ۵۰٪ رسید.

الف) IPv6 هنوز کامل نشده

ب) IPv6 کندتر است

ج) کسی نیست که روز کوچ را تعیین کند

د) قانون جلویش را گرفته

چرا IPv6 را نمی‌شود مثل ۱۹۸۳ یک‌شبه جایگزین کرد؟

IPv6، نسخهٔ تازهٔ نشانی‌های اینترنت، از ۱۹۹۸ استاندارد است؛ سهمش در آمار Google تازه اردیبهشت ۱۴۰۵ به ۵۰٪ رسید.

الف) IPv6 هنوز کامل نشده	**ب)** IPv6 کندتر است	**ج)** کسی نیست که روز کوچ را تعیین کند	**د)** قانون جلویش را گرفته

پاسخ: ج) در ۱۹۸۳ یک متولی برای شبکه‌ای کوچک تاریخ گذاشت؛ امروز هزاران اپراتور مستقل هست و هرکدام هزینهٔ کوچ خودش را می‌دهد، و NAT (چند دستگاه پشت یک نشانی) کمبود را موقتاً پوشانده است (APNIC).

شبکهٔ بی‌مرکز را هیچ‌کس به‌تنهایی اداره نمی‌کند؛ پس با یک دستور هم نمی‌شود عوضش کرد.

بخش ۲ از ۷

تولد وب

تا اینجا: اینترنت برای ایمیل و انتقال فایل بود؛ پیدا کردن اطلاعات سخت بود.

سؤال این بخش

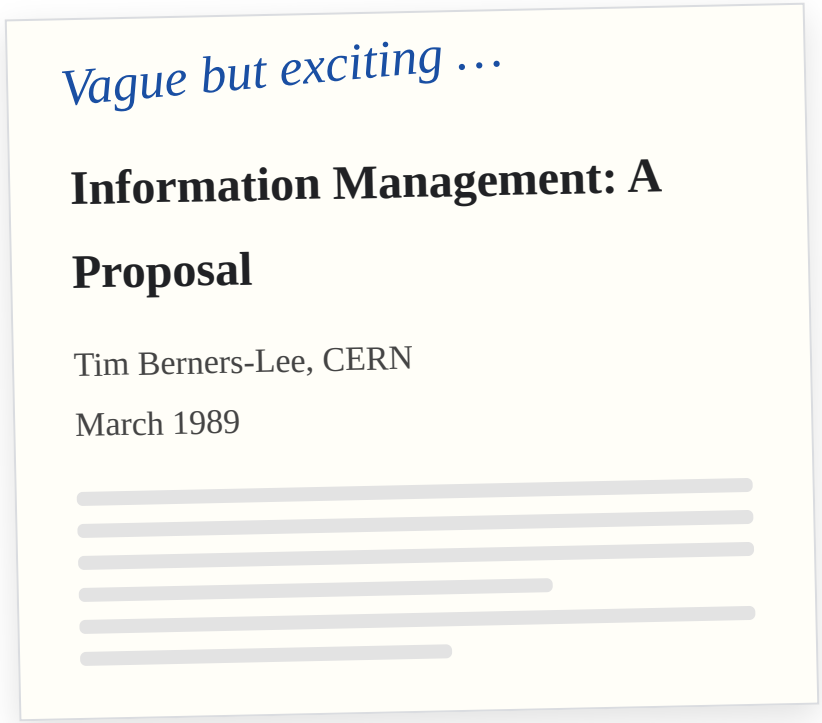
چطور سندهای پراکنده روی هزاران کامپیوتر را به هم وصل کنیم؟

00-how-we-got-here.todo

- [x] ۱۹۶۹ تا ۱۹۸۳
- [>] ۱۹۸۹ تا ۱۹۹۳ ◀
- [ ] ۱۹۹۴ تا ۲۰۰۱
- [ ] ۲۰۰۴ تا ۲۰۱۲
- [ ] ۲۰۱۱ تا ۲۰۲۲
- [ ] ۲۰۲۲ تا امروز
- [ ] از اینجا به بعد

«مبهم ولی هیجان‌انگیز»

- مارس ۱۹۸۹: تیم برنرز-لی در CERN پیشنهادی برای مدیریت اطلاعات نوشت
- رئیسش، مایک سندال، بالای آن نوشت **«مبهم ولی هیجان‌انگیز»** و اجازهٔ ادامهٔ کار را داد
- این پیشنهاد دو سال بعد به وب جهان‌گستر تبدیل شد



بازسازی صفحهٔ اول؛ اسکن اصلی در CERN و متن کامل

وب یک پروژهٔ جانبی بود که یک مدیر اجازهٔ ادامه‌اش را داد.

سه اختراع در یک پروژه

- URL: نشانی یکتا برای هر سند، روی هر کامپیوتری (جلسهٔ ۱)
- HTTP: قرارداد درخواست و پاسخ برای گرفتن سند (جلسهٔ ۲)
- HTML: قالب سند، با پیوند به سندهای دیگر
- نخستین وب‌سایت در ۱۹۹۱ منتشر شد و هنوز روی info.cern.ch باز می‌شود



نخستین وب‌سرور؛ روی برچسبش نوشته‌اند: «خاموشش نکنید!»
منبع: User:Coolcaesar at en.wikipedia · CC BY-SA 3.0

وب یک برنامه روی اینترنت است، مثل ایمیل؛ و دو تا از این سه اختراع موضوع همین درس‌اند.

چرا وب برنده شد و Gopher نه؟

در ۱۹۹۳ Gopher، سامانهٔ منویی دانشگاه مینه‌سوتا برای پیدا کردن سند، از وب محبوب‌تر بود.

الف) سرعت بیشتر وب

ب) نمایش تصویر در وب

ج) مجوز: Gopher پولی شد، وب رایگان

د) پشتیبانی یک شرکت بزرگ

چرا وب برنده شد و Gopher نه؟

در ۱۹۹۳ Gopher، سامانهٔ منویی دانشگاه مینه‌سوتا برای پیدا کردن سند، از وب محبوب‌تر بود.

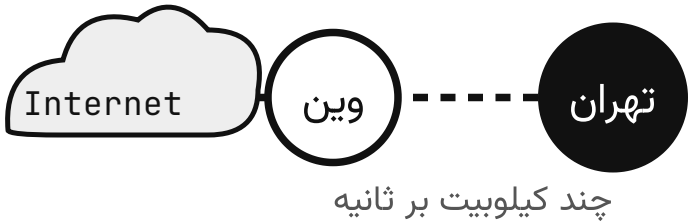
الف) سرعت بیشتر وب	**ب)** نمایش تصویر در وب	**ج)** مجوز: Gopher پولی شد، وب رایگان	**د)** پشتیبانی یک شرکت بزرگ

پاسخ: ج) در ۱۹۹۳ دانشگاه مینه‌سوتا برای استفادهٔ تجاری از سرور Gopher هزینهٔ مجوز خواست؛ در ۳۰ آوریل همان سال CERN نرم‌افزار وب را رایگان و برای همه اعلام کرد. تصویر هم کمک کرد، ولی نقطهٔ برگشت مجوز بود.

> **در پروتکل‌ها، هزینهٔ پذیرش اغلب از بهتر بودن مهم‌تر است.**

ایران کِی به اینترنت وصل شد؟

- دی ۱۳۷۱ (ژانویهٔ ۱۹۹۳): پژوهشگاه دانش‌های بنیادی (IPM) با یک خط اجاره‌ای **baud ۹۶۰۰** به دانشگاه وین وصل شد؛ نخست روی شبکهٔ علمی BITNET و برای ایمیل (NSRC)
- سپس همین خط اتصال کامل اینترنت شد و ایران ۵۰۰ نشانی IP گرفت؛ با این سرعت، دریافت یک عکس یک‌مگابایتی حدود ربع ساعت طول می‌کشید
- فروردین ۱۳۷۳: دامنهٔ `.ir` با متولی‌گری IPM ثبت شد، و از بهار همان سال ندا رایانه، با پشتیبانی شهرداری تهران، ایمیل اینترنتی را به عموم عرضه کرد



> **از روز اول، مسیر بین‌الملل برای ما یک تصمیم مهندسی بوده است، نه یک فرض.**

بخش ۳ از ۷

جنگ اول مرورگرها

تا اینجا: وب رایگان شد؛ حالا کسی باید آن را به خانه‌ها می‌برد.

سؤال این بخش

وقتی یک شرکت مرورگر بازار را می‌گیرد، چه بر سر استانداردها می‌آید؟

00-how-we-got-here.todo

- [x] ۱۹۶۹ تا ۱۹۸۳
- [x] ۱۹۸۹ تا ۱۹۹۳
- [>] **۱۹۹۴ تا ۲۰۰۱** ◀
- [ ] ۲۰۰۴ تا ۲۰۱۲
- [ ] ۲۰۱۱ تا ۲۰۲۲
- [ ] ۲۰۲۲ تا امروز
- [ ] از اینجا به بعد

کدام مرورگر وب را برای همه ساخت؟

- (۱۹۹۳) Mosaic اولین مرورگر پرطرفداری بود که تصویر را کنار متن نشان می‌داد
- سازندگانش Netscape را ساختند؛ عرضهٔ سهام آن در اوت ۱۹۹۵ آغاز تب دات‌کام بود
- وب از ابزار دانشگاهی به **محصولی برای همه** تبدیل شد

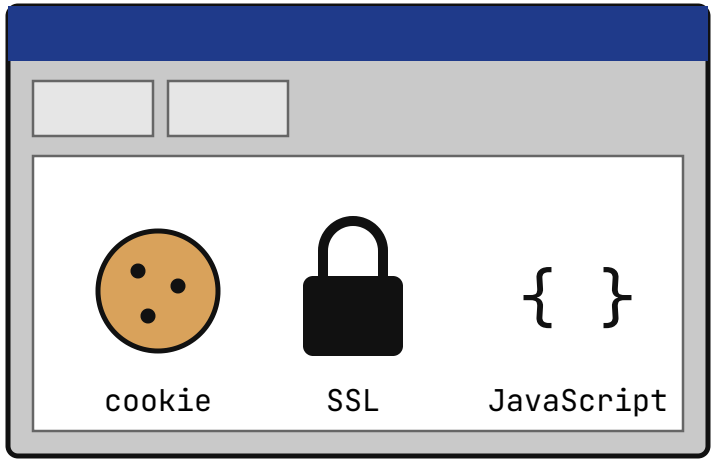


مرورگر NCSA Mosaic: متن و تصویر در یک صفحه

منبع: Charles Severance · CC0

رابط کاربری خوب، یک پروتکل را از آزمایشگاه به خانه‌ها برد.

سه اختراع Netscape که هنوز با ماست



- کوکی (۱۹۹۴): راهی برای اینکه برنامهٔ وب، روی پروتکلی stateless، کاربر را به یاد بیاورد (جلسهٔ ۳)
- SSL (۱۹۹۵): ارتباط مرورگر و سرور را رمز می‌کند، تا شمارهٔ کارت در خرید اینترنتی بین راه خوانده نشود؛ نوادهٔ امروزی‌اش TLS است (جلسهٔ ۴)
- JavaScript (۱۹۹۵): برنامه‌ای که **داخل صفحه اجرا می‌شود**؛ نسخهٔ اولش در حدود ده روز نوشته شد

هر سه برای حل یک مشکل تجاری فوری ساخته شدند و هر سه امروز استاندارد جهانی‌اند.

چرا `flatten` در JavaScript به `flat` تغییر نام داد؟

بهار ۱۳۹۷ قرار بود آرایه‌های JavaScript متد `flatten` بگیرند؛ در آخرین لحظه نامش `flat` شد.

الف) `flat` کوتاه‌تر بود

ب) تداخل با پیاده‌سازی‌های قدیمی

ج) `flatten` ثبت تجاری داشت

د) هماهنگی با Python

چرا `flatten` در JavaScript به `flat` تغییر نام داد؟

بهار ۱۳۹۷ قرار بود آرایه‌های JavaScript متد `flatten` بگیرند؛ در آخرین لحظه نامش `flat` شد.

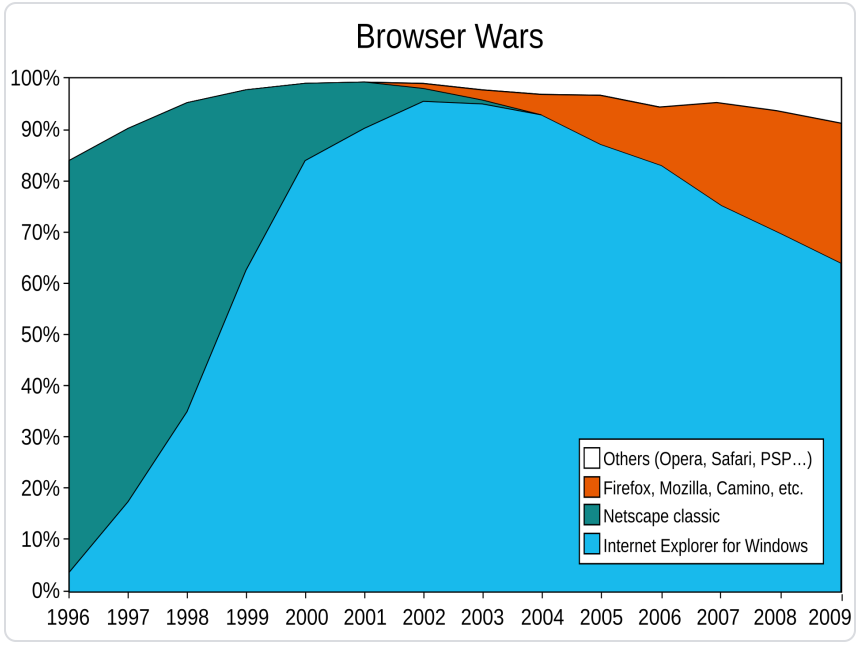
الف) `flat` کوتاه‌تر بود	**ب)** تداخل با پیاده‌سازی‌های قدیمی	**ج)** `flatten` ثبت تجاری داشت	**د)** هماهنگی با Python

پاسخ: ب) کتابخانهٔ قدیمی MooTools خودش `flatten` را به آرایه‌ها اضافه کرده بود و نسخهٔ استاندارد سایت‌هایی را که دیگر کسی به‌روزشان نمی‌کرد می‌شکست؛ کمیته نام را عوض کرد، نه وب را (SmooshGate).

> **«وب را نشکن»: تصمیمی که در وب پخش شد، پس گرفته نمی‌شود.** `#Backward_Compatibility`

Microsoft چطور جواب داد؟

- Windows روی بیش از ۹۰٪ رایانه‌های شخصی بود؛ Microsoft مرورگرش، Internet Explorer، را **رایگان و از پیش نصب‌شده** در آن گذاشت
- قراردادها هم راه رقیب را بستند: سازندگان رایانه اجازه نداشتند آیکون IE را بردارند
- سهم IE در اوایل دههٔ ۲۰۰۰ از ۹۰٪ گذشت؛ دادگاه این را **حفظ غیرقانونی انحصار Windows** دانست (پروندهٔ ضدانحصار)، و Netscape در ۱۹۹۸ کدش را متن‌باز کرد؛ بذر Firefox



سهم بازار مرورگرها: سقوط Netscape و اوج IE

منبع: Wereon · CC BY-SA 3.0

اهرم Microsoft انحصار Windows بود، نه کیفیت مرورگر: وقتی رقابت بر سر توزیع باشد، بهترین محصول لزوماً برنده نمی‌شود.

وقتی یک مرورگر برنده شد، چه اتفاقی افتاد؟

- IE6 در ۲۰۰۱ منتشر شد و نسخهٔ بعدی، IE7، **پنج سال بعد** در ۲۰۰۶ آمد
- توسعه‌دهندگان برای رفتارهای خاص یک مرورگر کد می‌نوشتند، نه برای استاندارد
- جملهٔ «این سایت فقط با Internet Explorer درست دیده می‌شود» روی صفحه‌ها رایج شد

انحصار استاندارد را متوقف می‌کند؛ رفتار یک پیاده‌سازی جای مشخصات را می‌گیرد.

بخش ۴ از ۷

وب دوباره باز می‌شود

تا اینجا: IE6 سال‌ها بی‌رقیب ماند و استانداردها ایستادند.

سؤال این بخش

چطور می‌شود وبی را که در یک مرورگر قفل شده، دوباره باز کرد؟

00-how-we-got-here.todo

- [x] ۱۹۶۹ تا ۱۹۸۳
- [x] ۱۹۸۹ تا ۱۹۹۳
- [x] ۱۹۹۴ تا ۲۰۰۱
- [>] **۲۰۰۴ تا ۲۰۱۲** ◀
- [ ] ۲۰۱۱ تا ۲۰۲۲
- [ ] ۲۰۲۲ تا امروز
- [ ] از اینجا به بعد

چه کسی استاندارد را از رفتار واقعی مرورگرها نوشت؟

- (۲۰۰۴) Firefox، وارث متن‌باز Netscape، رقابت را دوباره زنده کرد
- همان سال Apple و Mozilla و Opera گروه WHATWG را ساختند تا استاندارد را **از روی رفتار واقعی مرورگرها** بنویسند
- استاندارد URL که در جلسهٔ ۱ می‌بینیم، کار همین گروه است

> **استانداردی که با رفتار واقعی نخواند، فقط روی کاغذ استاندارد است.**

چرا Chrome خودش را Mozilla هم معرفی می‌کند؟

هدر `User-Agent` که Chrome 143 روی Windows در هر درخواست HTTP می‌فرستد (جلسهٔ ۲)، در اصل یک خط:

```
Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
    (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36
```

چرا Chrome خودش را Mozilla هم معرفی می‌کند؟

هدر `User-Agent` که Chrome 143 روی Windows در هر درخواست HTTP می‌فرستد (جلسهٔ ۲)، در اصل یک خط:

```
Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
    (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36
```

پاسخ: سایت‌های دههٔ ۹۰ صفحهٔ کامل را فقط به Netscape می‌دادند، که خودش را Mozilla می‌نامید؛ IE خودش را سازگار با Mozilla معرفی کرد و هر مرورگر بعدی ادای قبلی را درآورد (تاریخچه). کد صفحه هم اگر با `navigator.appName` نام مرورگر را بپرسد، امروز **هر مرورگری جواب می‌دهد** `"Netscape"`.

> **وقتی سایت به‌جای قابلیت، نام مرورگر را بسنجد، هر مرورگر تازه مجبور به دروغ می‌شود.**

وب چطور از «سند» به «برنامه» رسید؟

- تا آن روز هر کلیک یعنی یک صفحهٔ تازه: مرورگر کل صفحه را دوباره از سرور می‌گرفت
- (۲۰۰۴) Gmail و (۲۰۰۵) Google Maps **فقط دادهٔ تازه را در پس‌زمینه می‌گرفتند** و همان صفحه را به‌روز می‌کردند؛ ابزارش XMLHttpRequest بود که Microsoft برای IE ساخته بود، و این شیوه در ۲۰۰۵ Ajax نام گرفت
- اما هنوز فقط مرورگر می‌توانست گفتگو را شروع کند؛ برای اینکه **سرور هم بتواند اول حرف بزند**، WebSocket در ۲۰۱۱ آمد (جلسهٔ ۶)

> **وقتی صفحه به برنامه تبدیل شد، الگوی «هر کلیک، یک صفحهٔ تازه» دیگر کافی نبود و پروتکل هم باید عوض می‌شد.**

وب چطور به جیب رسید؟

- (۲۰۰۷) iPhone یک مرورگر کامل را به گوشی آورد؛ چند سال بعد ترافیک وب گوشی‌ها از رایانه‌های رومیزی بیشتر شد و **کندی شبکه** دوباره مسئلهٔ اصلی شد
- (۲۰۰۸) Chrome هر زبانه را در پردازه‌ای جدا اجرا کرد و **دو موتور** دارد: موتور مرورگر که از HTML و CSS صفحه را می‌سازد و رسم می‌کند، و موتور JavaScript به نام V8 که کد صفحه را اجرا می‌کند
- موتور مرورگر Chrome اول WebKit بود، همان موتور Safari؛ Google در ۲۰۱۳ آن را منشعب کرد و Blink نامید



نخستین iPhone (۲۰۰۷): مرورگر کامل در جیب
منبع: Carl Berkeley from Riverside California · CC BY-SA 2.0

وب از میز کار به جیب رفت و پروتکل‌ها باید با شبکه‌های کند و ناپایدار کنار می‌آمدند.

امروز بیشتر صفحه‌های وب را کدام موتور مرورگر نشان می‌دهد؟

الف) Blink (Chromium)

ب) Gecko (Firefox)

ج) WebKit (Safari)

د) Trident (IE)

امروز بیشتر صفحه‌های وب را کدام موتور مرورگر نشان می‌دهد؟

الف) Blink (Chromium)	ب) Gecko (Firefox)	ج) WebKit (Safari)	د) Trident (IE)

پاسخ: الف) Chrome و Edge و Opera و Brave، و حتی مرورگرهای هوش مصنوعی مثل Comet و ChatGPT Atlas، همه روی Chromium، پروژهٔ متن‌باز Google، ساخته شده‌اند؛ Edge در ۲۰۲۰ موتور خودش را کنار گذاشت. استثنا iPhone است: قاعدهٔ Apple هر مرورگری را، حتی Chrome، به استفاده از WebKit مجبور می‌کند، مگر در اتحادیهٔ اروپا و ژاپن که قانون موتور دیگر را مجاز کرده است. در ۱۱ شهریور ۱۴۰۴ دادگاهی در آمریکا حکم به فروش اجباری Chrome نداد و یکی از دلیل‌هایش رقابت تازهٔ هوش مصنوعی بود (NPR).

> **جنگ دوم با برندهٔ تازه تمام شد؛ پرسش امروز این است که آیا داستان IE6 تکرار می‌شود.**

بخش ۵ از ۷

وب امن و سریع

تا اینجا: وب همه‌جا بود، ولی بیشترش رمز نشده بود.

سؤال این بخش

وقتی بفهمیم کسی گوش می‌دهد، پروتکل‌ها چه باید بکنند؟

00-how-we-got-here.todo

- [x] ۱۹۶۹ تا ۱۹۸۳
- [x] ۱۹۸۹ تا ۱۹۹۳
- [x] ۱۹۹۴ تا ۲۰۰۱
- [x] ۲۰۰۴ تا ۲۰۱۲
- [>] **۲۰۱۱ تا ۲۰۲۲** ◀
- [ ] ۲۰۲۲ تا امروز
- [ ] از اینجا به بعد

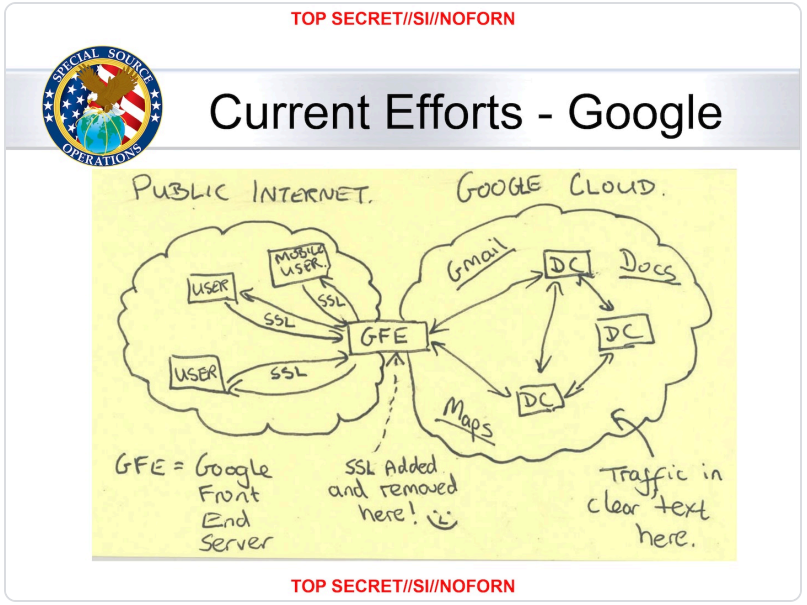
سناریوی واقعی: گواهی جعلی Gmail در ایران

- گواهی را یک CA، یعنی صادرکنندهٔ مورد اعتماد، امضا می‌کند تا مرورگر بداند با خود `google.com` حرف می‌زند (جلسهٔ ۴)
- تیر ۱۳۹۰ نفوذگری از DigiNotar هلندی گواهی جعلی `*.google.com` گرفت؛ به گزارش Fox-IT حدود ۳۰۰ هزار نشانی IP که ۹۹٪ از ایران بودند با آن به میانجی وصل شدند
- ۵ شهریور ۱۳۹۰ کاربری در ایران گزارش داد Chrome اتصال به Gmail را قطع می‌کند: Chrome برای Google **فقط گواهی‌های مشخصی را می‌پذیرفت** (certificate pinning)

> **یک CA خراب کافی بود؛ پاسخ صنعت Certificate Transparency شد: هر گواهی در فهرستی عمومی ثبت می‌شود.**

وقتی معلوم شد چه کسی گوش می‌دهد

- در ۲۰۱۳ اسناد اسنودن نشان داد شنود گستردهٔ اینترنت واقعی است
- رمزنگاری Google فقط تا لبهٔ شبکه‌اش بود و ترافیک بین مراکز داده‌اش **بدون رمز** شنود می‌شد (MUSCULAR)؛ Google و Yahoo بعد از آن این ترافیک را هم رمز کردند
- نهاد استانداردهای اینترنت، IETF، در RFC 7258 اعلام کرد **شنود فراگیر یک حمله است**؛ پس از آن پروتکل‌های تازه مثل QUIC رمزنگاری را اجباری کردند



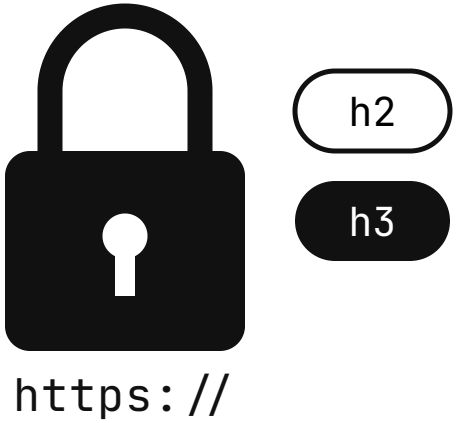
سند NSA در ۲۰۱۳: «SSL added and removed here»

منبع: U.S. National Security Agency · Public domain

امنیت از یک افزونه به یک پیش‌فرض تبدیل شد.

HTTPS چطور همه‌گیر شد؟

- Let’s Encrypt از ۲۰۱۵ گواهی HTTPS را **رایگان و خودکار** کرد
- از Chrome 68 در مرداد ۱۳۹۷، هر صفحهٔ http با برچسب «Not secure» نشان داده می‌شود
- از ۲۴ اسفند ۱۴۰۴ عمر گواهی حداکثر ۲۰۰ روز است و تا ۲۰۲۹ به ۴۷ روز می‌رسد؛ تمدید دستی عملاً ناممکن و **خودکارسازی اجباری** می‌شود



وقتی کار درست آسان و کار نادرست قابل‌دیدن شود، رفتار همه عوض می‌شود.

چطور HTTP را بدون شکستن وب بازنویسی کنیم؟

- Google در ۲۰۰۹ پروتکل آزمایشی SPDY را روی Chrome و سرورهای خودش امتحان کرد
- SPDY پایهٔ HTTP/2 در ۲۰۱۵ شد، و آزمایش بعدی، QUIC، پایهٔ HTTP/3 در ۲۰۲۲
- **معنای HTTP ثابت ماند** و فقط قالب انتقال عوض شد؛ مرورگر و سرور نسخه را با هم توافق می‌کنند و اگر یکی بلد نبود به HTTP/1.1 برمی‌گردند، پس سایت‌ها مجبور به تغییر نشدند (جلسهٔ ۴)

> **پروتکلی که معنایش را از قالب انتقالش جدا کند، می‌تواند بدون شکستن گذشته تغییر کند.**

چرا HTTP/3 روی UDP ساخته شد، نه روی یک پروتکل انتقال تازه؟

TCP و UDP دو پروتکل انتقال اصلی اینترنت‌اند؛ HTTP/3 روی QUIC کار می‌کند و QUIC روی UDP.

الف) UDP ذاتاً از TCP سریع‌تر است

ب) پروتکل تازه از فایروال رد نمی‌شود

ج) TCP منسوخ شده است

د) UDP خودش رمزنگاری دارد

چرا HTTP/3 روی UDP ساخته شد، نه روی یک پروتکل انتقال تازه؟

TCP و UDP دو پروتکل انتقال اصلی اینترنت‌اند؛ HTTP/3 روی QUIC کار می‌کند و QUIC روی UDP.

الف) UDP ذاتاً از TCP سریع‌تر است	**ب)** پروتکل تازه از فایروال رد نمی‌شود	**ج)** TCP منسوخ شده است	**د)** UDP خودش رمزنگاری دارد

پاسخ: ب) بیشتر فایروال‌ها و NATها بستهٔ هر پروتکلی جز TCP و UDP را دور می‌ریزند (ossification)؛ پس QUIC **بسته‌هایش را درون UDP می‌فرستد**. بهتر کردن TCP کند است، چون در هستهٔ سیستم‌عامل است؛ QUIC درون مرورگر است (مقالهٔ طراحان) و بیشتر header خودش را رمز می‌کند تا این بار کسی به جزئیاتش وابسته نشود.

> **هرچه تجهیزات بیشتری به شکل امروز یک پروتکل وابسته شوند، تغییر دادنش سخت‌تر می‌شود.**

بخش ۶ از ۷

عصر هوش مصنوعی

تا اینجا: وب امن و سریع شد؛ حالا خوانندهٔ تازه‌ای از راه رسید.

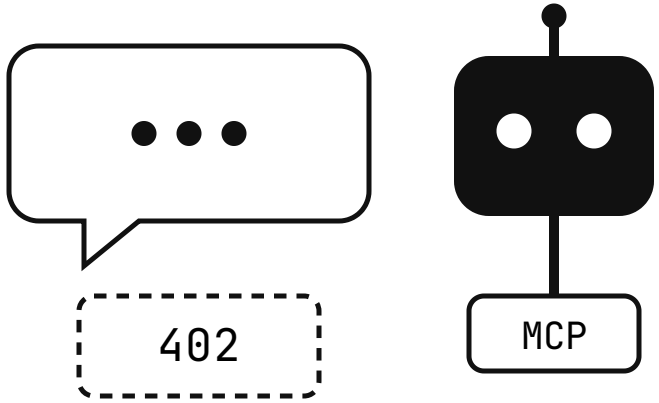
سؤال این بخش

وقتی کلاینت دیگر انسان نیست، چه چیزی در پروتکل‌ها عوض می‌شود؟

00-how-we-got-here.todo

[x] ۱۹۶۹ تا ۱۹۸۳
[x] ۱۹۸۹ تا ۱۹۹۳
[x] ۱۹۹۴ تا ۲۰۰۱
[x] ۲۰۰۴ تا ۲۰۱۲
[x] ۲۰۱۱ تا ۲۰۲۲
[>] ۲۰۲۲ تا امروز ▶
[ ] از اینجا به بعد

گفتگو با یک مدل، رابط تازهٔ وب



- ChatGPT در آذر ۱۴۰۱ منتشر شد و گفتگو با مدل زبانی را همه‌گیر کرد
- مدل متن را توکن‌به‌توکن تولید می‌کند و API هر تکه را همان لحظه با SSE می‌فرستد؛ برای همین پاسخ کم‌کم ظاهر می‌شود (جلسهٔ ۶)
- مدل‌ها برای یادگیری و پاسخ دادن **وب را می‌خوانند**، و این رابطهٔ سایت‌ها با خواننده‌هایشان را عوض کرد

> **خوانندهٔ وب دیگر فقط انسان نیست.**

امروز چه کسی بیشتر وب را می‌خواند: انسان یا بات؟

الف) انسان، با فاصلهٔ زیاد

ب) تقریباً نصف‌نصف، با برتری انسان

ج) بات‌ها، کمی بیشتر از نصف

د) بات‌ها، بیش از ۹۰ درصد

امروز چه کسی بیشتر وب را می‌خواند: انسان یا بات؟

الف) انسان، با فاصلهٔ زیاد	**ب)** تقریباً نصف‌نصف، با برتری انسان	**ج)** بات‌ها، کمی بیشتر از نصف	**د)** بات‌ها، بیش از ۹۰ درصد

پاسخ: ج) در ۲۰۲۴ برای اولین بار در یک دهه ترافیک خودکار از انسان بیشتر شد (۵۱ درصد)، و طبق گزارش ۱۴۰۵ Imperva در ۲۰۲۵ به ۵۳ درصد رسید؛ agentهای هوش مصنوعی، یعنی برنامه‌هایی که به‌جای کاربر وب را می‌گردند و کار انجام می‌دهند، حالا دستهٔ تازه‌ای از این ترافیک‌اند.

> **پروتکلی که برای انسان طراحی شد، حالا بیشتر با ماشین حرف می‌زند.**

سایت‌ها با خزنده‌های هوش مصنوعی چه می‌کنند؟

- فایل robots.txt از ۱۹۹۴ هست و از ۲۰۲۲ با RFC 9309 استاندارد شد؛ ولی هنوز فقط یک «خواهش» است، نه قفل
- در تیر ۱۴۰۴ Cloudflare برای دامنه‌های تازه خزنده‌های هوش مصنوعی را **پیش‌فرض مسدود کرد**؛ در تیر ۱۴۰۵ بات جستجو و agent و آموزش را از هم جدا کرد
- کد وضعیت 402، «پرداخت لازم است»، نزدیک سه دهه بی‌استفاده ماند تا x402: سرور 402 و قیمت را برمی‌گرداند و agent می‌پردازد؛ AWS از خرداد ۱۴۰۵ آن را ارائه می‌دهد (جلسهٔ ۷)

> **یک کد وضعیت فراموش‌شده، ابزار یک مدل اقتصادی تازه شد.**

وقتی بات هم خودش را Chrome معرفی می‌کند

- مرداد ۱۴۰۴ Cloudflare گزارش داد خزندهٔ اعلام‌نشدهٔ Perplexity، وقتی مسدود می‌شد، با این User-Agent روزی میلیون‌ها درخواست می‌فرستاد؛ Perplexity ادعا را رد کرد:

```
Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36
    (KHTML, like Gecko) Chrome/124.0.0.0 Safari/537.36
```

- خود Chrome از ۱۴۰۱ جزئیات User-Agent را ثابت کرد، مثل نسخهٔ دقیق مرورگر و سیستم‌عامل، و اصلشان را به هدرهای `Sec-CH-UA-*` برد
- راه تازه **هویت با امضا، نه با ادعا** است: Web Bot Auth در IETF (جلسهٔ ۷)

> **هدری که هرکس می‌تواند بنویسد، هویت نیست؛ همان درس دههٔ ۹۰، این بار برای بات‌ها.**

وقتی agent به‌جای شما مرور می‌کند

- مرورگرهای هوش مصنوعی مثل Comet (تیر ۱۴۰۴) و ChatGPT Atlas (مهر ۱۴۰۴) خودشان صفحه باز می‌کنند، فرم پر می‌کنند و حتی خرید می‌کنند؛ از بهمن ۱۴۰۴ خود Chrome هم با auto browse همین کار را می‌کند
- MCP (آذر ۱۴۰۳) پروتکلی است که agentها با آن به ابزارها و داده‌ها وصل می‌شوند
- پژوهشگران امنیتی در همان ماه‌های اول، prompt injection را در مرورگر Comet نشان دادند؛ **صفحه‌ای که agent می‌خواند، می‌تواند به او دستور بدهد** (جلسهٔ ۱)

> **کلاینت تازه، حمله‌های تازه می‌آورد؛ ولی روی همان پروتکل‌های قدیمی.**

تاریخ تکرار می‌شود: MCP چه چیزی را دوباره کشف کرد؟

- مشخصات تازهٔ MCP (مرداد ۱۴۰۵) session، یعنی حافظهٔ سرور از گفتگو، را حذف کرد: **stateless** مثل HTTP
- state به برنامه رفت: سرور شناسه‌ای می‌دهد و مدل آن را پس می‌فرستد، مثل کوکی (جلسهٔ ۳)
- پاسخ‌ها زمان اعتبار گرفتند، شبیه `max-age` در `Cache-Control` (جلسهٔ ۵)
- هدر `Mcp-Method` آمد تا واسطه‌ها بدون خواندن بدنهٔ درخواست مسیریابی کنند (جلسهٔ ۷)

> **پروتکل‌های تازه اغلب همان درس‌های قدیمی را دوباره یاد می‌گیرند؛ این درس برای این است که شما آن‌ها را از قبل بدانید.**

بخش ۷ از ۷

از اینجا به بعد

تا اینجا: از «LO» تا agentها را دیدیم.

سؤال این بخش

این داستان به کجا می‌رود، و چه الگوهایی از آن برای مهندسی می‌ماند؟

00-how-we-got-here.todo

[x] ۱۹۶۹ تا ۱۹۸۳
[x] ۱۹۸۹ تا ۱۹۹۳
[x] ۱۹۹۴ تا ۲۰۰۱
[x] ۲۰۰۴ تا ۲۰۱۲
[x] ۲۰۱۱ تا ۲۰۲۲
[x] ۲۰۲۲ تا امروز
[>] از اینجا به بعد ▶

وب ۱ و ۲ و ۳: نسخهٔ پروتکل یا برچسب؟

نسل	دوره	کاربر چه می‌کند؟	داده و هویت پیش کیست؟	نمونه
وب ۱	دههٔ ۱۹۹۰	می‌خواند	صاحب هر سایت	صفحه‌های ایستا
وب ۲	از ۲۰۰۴	می‌خواند و می‌نویسد	**چند پلتفرم بزرگ**	Gmail و شبکه‌های اجتماعی
وب ۳	از ۲۰۱۴	می‌خواند، می‌نویسد و «مالک» است	blockchain و کلید خود کاربر	Ethereum

این شماره‌ها را هیچ نهاد استانداردی نگذاشته است. Web 2.0 با کنفرانسی در ۲۰۰۴ سر زبان‌ها افتاد و «وب ۱» بعداً در برابرش ساخته شد. Web3 را Gavin Wood، از بنیان‌گذاران Ethereum، در ۲۰۱۴ پیشنهاد کرد: وبی روی blockchain، دفتر کل مشترکی که صاحب واحدی ندارد.

> **این شماره‌ها مثل HTTP/2 و HTTP/3 نسخهٔ پروتکل نیستند؛ برچسبی‌اند برای اینکه چه کسی می‌نویسد و داده پیش چه کسی می‌ماند.**

آیندهٔ وب: دادهٔ شما پیش چه کسی می‌ماند؟

- **وب ۳:** مالکیت روی blockchain ثبت می‌شود، نه نزد یک شرکت؛ ولی دسترسی به آن **از چند واسطهٔ متمرکز** می‌گذرد: NFT (سند مالکیت دیجیتال) سازندهٔ Signal در دی ۱۴۰۰ با حذف از یک سایت، از کیف پول خودش هم ناپدید شد
- **وب فدرال:** سرورهای مستقل با یک پروتکل مشترک، مثل ایمیل: ActivityPub زیر Mastodon، استاندارد رسمی وب از ۲۰۱۸، و Solid که برنرز-لی با آن داده را از برنامه جدا می‌کند
- **وب agentها:** کلاینت به‌جای شما عمل می‌کند؛ Chrome از بهمن ۱۴۰۴ WebMCP را می‌آزماید تا سایت کارهایش را **مستقیم به agent عرضه کند**

> **هر سه هنوز از همان DNS و TLS و HTTP می‌گذرند: پروتکل‌های این درس زیر هر سه‌اند.**

چهار الگو در همهٔ این داستان‌ها

الگو	نمونه‌ها
هزینهٔ پذیرش کمتر معمولاً برنده می‌شود	وب رایگان در برابر Gopher، و IE همراه Windows و Let’s Encrypt
انحصار و تمرکز شکننده است	IE6 و Chromium و قطعی AWS در مهر ۱۴۰۴
هرچه پرکاربردتر، عوض کردنش سخت‌تر	IPv6 و SmooshGate و User-Agent و QUIC روی UDP
امنیت دیرهنگام گران است	DigiNotar؛ و این بار زودتر، رمزنگاری پساکوانتومی

در ادامهٔ درس، هر جا یکی از این الگوها را دیدید علامتش بزنید.

برنامهٔ درس: در کلاس چه می‌کنیم و در تمرین چه؟

کجا	چه می‌کنیم
در کلاس	**مدل ذهنی و تصمیم مهندسی** : هر پروتکل چرا این شکلی است، و شما جای طراحش چه می‌کردید
در تمرین‌ها	**کار عملی** : همان را با ابزار واقعی، روی سیستم خودتان می‌بینید و می‌سازید
نمره	نیمی از نمره از تمرین و پروژه و ارائه است؛ جزئیاتش در سندی جدا در مخزن درس می‌آید

> **در کلاس می‌فهمیم «چرا»؛ در تمرین می‌بینیم و می‌سازیم «چطور».**

نقشهٔ فصل ۱: کجای راهیم؟

ch1/syllabus.todo

- [ ] جلسهٔ ۱: از URL تا سرور — URL و Origin و DNS، و TLS از دور
- [ ] جلسهٔ ۲: یک تراکنش HTTP — متد و کد وضعیت و header
- [ ] جلسهٔ ۳: state و هویت — cookie و CORS و CSRF
- [ ] جلسهٔ ۴: زیربنای HTTP — TCP و QUIC و TLS
- [ ] جلسهٔ ۵: واسطه‌ها — cache و proxy و CDN
- [ ] جلسهٔ ۶: فراتر از request/response — SSE و WebSocket
- [ ] جلسهٔ ۷: کلاینت‌های خودکار — agent و MCP

هر جلسه یک حلقه از همان زنجیرهٔ ابتدای فصل را بررسی می‌کند.

برای کنجکاوی بیشتر (۱ از ۲)

۱ چرا اولین پیام ARPANET بعد از دو حرف قطع شد، و در آن سال‌ها «از کار افتادن» یک گره چه معنایی داشت؟

سرنخ: روایت لئونارد کلاینراک از شب ۲۹ اکتبر ۱۹۶۹.

۲ چرا در بعضی سایت‌های فارسی جستجوی «کیک» نتیجهٔ «كيك» را پیدا نمی‌کند، با اینکه روی صفحه یکی به نظر می‌رسند؟

سرنخ: «ی» فارسی و «ي» عربی در Unicode دو نویسهٔ جدا هستند (`U+06CC` و `U+064A`)؛ استاندارد صفحه‌کلید فارسی ISIRI 9147 (فروردین ۱۳۸۶) را هم ببینید.

۳ RFC 7258 دقیقاً از طراحان پروتکل چه می‌خواهد؟

سرنخ: متن خود RFC را بخوانید؛ فقط چند صفحه است.

برای کنجکاوی بیشتر (۲ از ۲)

۴ **چرا یک خطای DNS در یک منطقهٔ AWS، سرویس‌هایی را در سراسر دنیا انداخت و چرا بازگشت کامل ساعت‌ها بیشتر از رفع خود خطا طول کشید؟**

سرنخ: بخش‌های DynamoDB و EC2 در گزارش رسمی AWS، و نقش منطقهٔ us-east-1.

۵ **مرورگر شما الان چه User-Agentای می‌فرستد، و Chrome کدام جزئیات را فقط وقتی می‌دهد که سرور بخواهد؟**

سرنخ: زبانهٔ Network در DevTools؛ هدرهای `Accept-CH` و `Sec-CH-UA` در MDN.

۶ **اصل Postel می‌گوید «در فرستادن سخت‌گیر و در پذیرفتن آسان‌گیر باش». چرا IETF در ۲۰۲۳ نوشت این اصل در بلندمدت به پروتکل‌ها آسیب می‌زند، و چه ربطی به IE6 و User-Agent دارد؟**

سرنخ: RFC 9413، و GREASE در RFC 8701.

منابع (۱ از ۲)

- گزارش AWS از قطعی ۲۸ مهر ۱۴۰۴ (20 Oct 2025) · تحلیل Pragmatic Engineer
- Smithsonian: LO · Paul Baran · ARPANET
- ISOC: Brief History of the Internet
- مقالهٔ TCP/IP (۱۹۷۴) · مقالهٔ end-to-end
- RFC 801 · Flag day (computing)
- APNIC: IPv6 در آمار Google
- info.cern.ch: نخستین وب‌سایت · پیشنهاد ۱۹۸۹
- History of the World Wide Web
- NSRC: Iran's Telecom and Internet Sector
- .ir · The Iranian: The Internet in Iran (۱۹۹۷)
- Pierre Omidyar · Browser wars
- یافته‌های دادگاه در پروندهٔ Microsoft در ۱۹۹۹ · رأی تجدیدنظر در ۲۰۰۱
- WHATWG · SmooshGate · تاریخچهٔ User-Agent
- Irancell: تاریخچه · NPR: حکم Chrome
- معرفی Blink از Chromium در ۲۰۱۳ · V8
- تصویرها: ویکی‌مدیا؛ صاحب اثر و مجوز زیر هر تصویر

منابع (۲ از ۲)

- Slate: DigiNotar · Fox-IT: Operation Black Tulip
- RFC 6962 · Certificate Transparency
- RFC 7258 · Washington Post: MUSCULAR
- DigiCert: گواهی ۴۷ روزه · Cloudflare: پساکوانتومی
- RFC 9114 · RFC 7540 · مقالهٔ QUIC
- Protocol ossification
- Imperva: Bad Bot Report 2026
- Cloudflare: ترافیک هوش مصنوعی (۱۴۰۵)
- InfoQ: x402 در Cloudflare و AWS
- RFC 9309 · پژوهش Brave دربارهٔ Comet
- قاعدهٔ WebKit در Apple · استثنای ژاپن
- Cloudflare: خزندهٔ Web Bot Auth · Perplexity
- TechCrunch: auto browse در Chrome
- MCP: مشخصات مرداد ۱۴۰۵
- Web3 · Web 2.0 · برداشت اول Moxie از web3
- Solid · ActivityPub · پیش‌نمایش WebMCP در Chrome